

A GitHub-based Data Collection Method for Software Defect Prediction

Jiayi Xu
School of Reliability and Systems
Engineering
Beihang University
Beijing 100191, China
hickeyhsu@buaa.edu.cn

Liang Yan
Beijing High Quality of Systems
Technology Co., Ltd
Beijing, China
ylbright@sina.com

Fei Wang
School of Reliability and Systems
Engineering
Beihang University
Beijing 100191, China
wangfeiabn@buaa.edu.cn

Jun Ai
School of Reliability and Systems
Engineering
Beihang University
Beijing 100191, China
*Correspondence:
aijun@buaa.edu.cn

Abstract—With the increasing scale and complexity of software systems, the defects of software are increasing every day. Software defect data is the foundation of research and application of software reliability. Currently, the lack of software defect data, its insufficient coverage, and the limits of the software types involved have become the bottleneck of software reliability research and application. Starting from GitHub, the open-source software hosting platform, this paper analyzes software defect data in open source projects and classifies the available software data. Based on the research of the GitHub and Git repository, we propose a defect data acquisition technology based on open-source software that uses pull requests as the breakthrough point of the method. Moreover, we advanced a software defect data preliminary treatment and built a software defect big datasets collecting system that contains fix-inducing change and contextual information of defects, which solves the class imbalance problem. According to this method, a software defect big data automatic acquisition platform based on GitHub was developed to realize the automatic collection of software defect data. Finally, the efficiency of data collection, correctness of data, and validity of the dataset application were verified by experiments. The results show that the proposed method is efficient and effective.

Keywords—Contextual Information, Data Collection, Defect Prediction, Fix-inducing Change, Open-Source Software, Software Defects

I. INTRODUCTION

Software defect data is the basis of software reliability research and application, including software defect prediction (SDP), which predicts the defect module of software based on historical data. Today, machine learning (ML) and deep learning are commonly applied in the software reliability field, which requires adequate data support. For a long time, the defect data used in the field of software reliability has mainly come from limited software test data and the analysis data of faults in use. Although such defect data are relatively complete, problems, such as inadequate amount, insufficient coverage, incomplete data information, and limited software type, have become the bottleneck of software reliability research and application.

At present, the MDP open dataset provided by NASA and the PROMISE dataset are the most widely used datasets in the field of SDP; however, these datasets all have defects, such as insufficient data, serious class imbalance, lack of semantic information, and lack of contextual information.

Recently, open-source software (OSS) has become popular all over the world. The largest open-source project site,

GitHub.com, now has tens of millions of OSS projects, which makes it possible for us to use the Internet to access huge amounts of software defect data[1]. Some researchers have proposed their methods of collecting software defect data based on the OSS platform. For example, Kapur[2] proposed a method of building a dataset with the help of the open-source repository on GitHub and the corresponding defect tracking system like Apache. Russell[3] used static analysis tools, such as Flawfinder, to label open source project data and build defect datasets

However, the current methods all have problems, such as insufficient data, poor coverage of defect types, lack of contextual information, and class imbalance. Not many software projects are managed by a defect tracking system. Static analysis tools, which are mainly based on rules, usually focus on syntax and ignore the defect caused by semantics and contextual information that are always related to the defect. Code semantics refers to the meaning of code statements, expressing the logic and function of the code. The contextual information refers to the descriptive information of the software project and the descriptive information of the defect, which expresses the function of the module where the software defect is located, its causes, and its manifestation.

In this paper, a novel approach is proposed to collect and properly label defect data from the biggest OSS platform, GitHub.com. After analyzing the main feature of software defect data and GitHub workflow, git repositories are queried to identify defect-related pull requests (PRs), and then obtain class-level samples from the workflow of the code changes. If a defect-related PR is labeled as fixed, then it is assumed that a defect would become a non-defect after fixing. Then, the related software module before the fix is treated as a defect sample and the one after is seen as a non-defect sample. Therefore, the number of defect samples always equals the number of non-defect samples, which means the class imbalance problem is solved. The fix-inducing change code is extracted through an abstract syntax tree (AST), and the contextual information from the PRs is extracted through natural language processing (NLP). An automatic defect data collecting system is built based on the method we proposed. The SDP is used to validate the dataset. The major contributions of this paper can be summarized as follows:

1) We propose a novel method to collect software defect data based on GitHub.com and label them using PRs, which solves the imbalanced problem.

2) We propose an approach to extract the exact defect code and contextual information from defect data.

This work was supported by funding 61400020407 and JSZL2017601C001.

3) We create a public defect dataset based on GitHub. The experimental results show that the dataset is valid for the application of defect prediction. The performance of defect prediction on our defect dataset is better than the traditional defect dataset.

II. BACKGROUND

A. Software defect prediction

With the extensive application of software in an increasing number of critical systems (such as systems in the financial industry, transportation industry, etc.), SDP has become an important topic in the field of software engineering. The purpose of SDP is to evaluate the defect tendency of the software system and software module. As the distribution of software defects follows the Pareto effect[4], 80% of software defects are typically only located in fewer than 20% of the system's files. Therefore, the efficiency and quality of software defect testing can be greatly improved by prioritizing the detection of modules with high software defect tendencies.

At present, SDP methods based on machine learning and deep learning have become mainstream[5]. The effect of this SDP method is mainly influenced by the software defect dataset and SDP model. This paper focuses on software defect datasets. As the input of the SDP model, the software defect dataset is an important support for SDP. Currently, public datasets, such as NASA's MDP dataset and the PROMISE dataset, are commonly used in SDP. However, these datasets generally have the problem of insufficient data volume, and the data only contains some metrics data of the class without important information like code semantics and contextual information.

B. GitHub

In the past decade, the trend of OSS that was initiated by the popularity of the Linux system has made it possible to obtain a huge amount of software defect data. GitHub, as a software source code hosting service for version control through Git, has become the largest OSS hosting platform in the world. According to the 2018 annual report released at the GitHub Universe developer conference, GitHub has over 31 million developers, over 96 million repositories, and over 200 million PR. It contains a large amount of software defect information and repair data, which makes it possible to acquire software defect big data, establish a software defect database, and provide big-data support for deep learning.

III. METHODOLOGY

This section describes the details of the proposed method. The overall framework of the proposed approach is shown in Fig. 1. The framework is mainly divided into four key steps. First, the range of the data that can be collected from GitHub is identified and it is analyzed as to which of them are related to defect. After figuring out what is desired to be accessed, data from the GitHub local repository data and the remote repository data are collected through PRs, which is a workflow of GitHub. For software information, defect-related PRs information, and version description, their topics are extracted as defect contextual information using NLP. For each defect-related PRs, the code before and after the fix is achieved to recognize the

exact fix-inducing change code based on AST. At last, a dataset that consists of the software defect code and the context is created by using the proposed method.

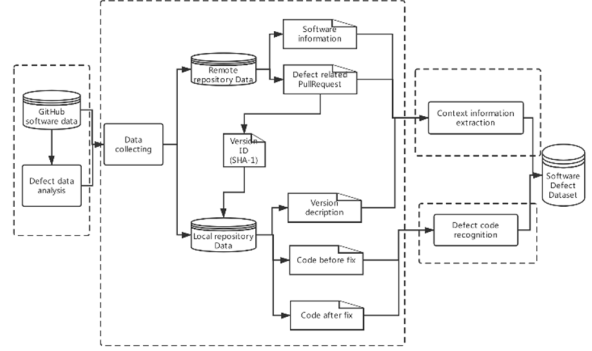


Fig. 1 The framework of the proposed method

A. Open-source software defect data analysis

First, the range of the data that can be collected needs to be determined. The available data range is mainly divided into two parts according to the source: remote repository data that can be obtained through the API provided by the GitHub platform and the local repository data that can be obtained through the local git project repository. By referring to the GitHub developer guide's documentation on GitHub API v4 to understand the meaning, content form, and access of each data, we mastered a total of 60 data collection methods in the remote repository and 28 data collection methods in the local repository.

After determining the range of available data, the software data that can be collected are classified and analyzed by referring to traditional software defect measurement elements and considering the requirements of existing SDP technologies based on a deep learning model. Finally, the three types of data to be collected are summarized, as shown in Fig. 2, totaling 15 kinds of data.

Data Name	type	Description
ProjectName	Project Information	The name of the project
ProjectOwner		The owner name of the project
Language		The programming language of the project
Git_Address		The git address of the project
Project Description		The description of the project provided by the owner
Project Label		The label of the project provided by the owner
PR Title	PRs information from remote repository	The title of the defect related PR
PR Topic		The defect topic we extract mainly from the PR information and project information
PR Description		The description of the defect related PR
SHA_New		The ID of the version after the defect being fixed
SHA_Old		The ID of the version before the defect being fixed
Path		The path of the changed files between the version before/after fixed
Content_New	Defect related code	The content of the related files after the defect being fixed
Content_Old		The content of the related files after the defect being fixed
Defect Code		The defect code we recognize from the content of the related files content

Fig. 2 Data to be collected

B. Software defect data collection based on Github

In the GitHub workflow, users report defects in the issue module of an open-source project. After locating the defect, the developer establishes the corresponding repair branch and opens a PR to initiate a review and notify the maintainers of the changes. Each PR can be labeled with ‘bug’, ‘enhancement’, ‘feature’, and so on. The branch is merged after being audited by the maintainers, and the PR is closed at the same time. The PR workflow of GitHub is shown in Fig. 3.

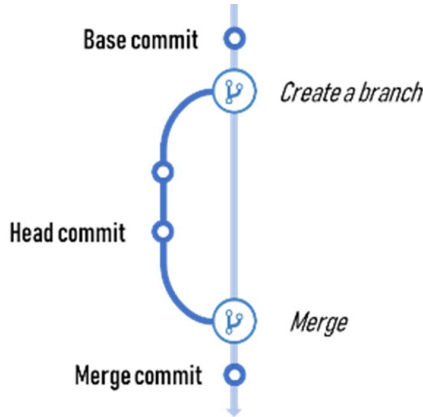


Fig. 3 PR workflow

We believe that software defect data collection on GitHub can be divided into the following four steps: project selection, identification of defect-related PR, remote repository data collection, and local repository data collection. The framework of the software defect data collection process based on GitHub is shown in Fig. 4.

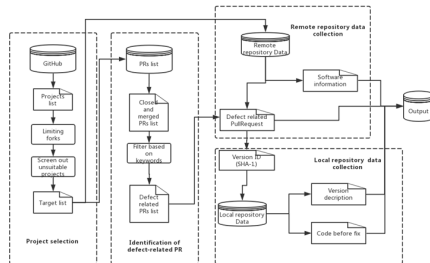


Fig. 4 Software defect data collection

1) Project selection

There are over 100 million software projects on GitHub, but not all of them are suitable targets for software defect collection. The software defect data collection target project shall meet the following conditions: 1) the project shall be a complete software; 2) the project shall undergo sufficient iterations to ensure that defects in the project software are identified and fixed as much as possible. Therefore, we first crawled the names, owners, and descriptions of Java software projects with more than 1,000 forks from GitHub. The reason for limiting the number of forks is that it was assumed that defects in software with more forks were more likely to be discovered.

Then, a simple screening was made according to the description of the software, and the projects whose description and PR were in Chinese, non-software projects (e.g., Snailclimb/JavaGuide is a guide of job interview), and other projects that were not suitable for collecting software defect data were removed, and a list of hot software items were obtained as the data collection target.

2) Identification of defect-related PR

There are often thousands of PRs in a project, but not all of them are defect related. In the GitHub workflow, a PR is also used for iteration of feature updates and for software changes that are not related to defects. The label and title of PR can be used as a quick way to identify whether a PR is related to software defects. The PRs can be filtered through keywords, PR switch state, and PR merge stats, as conditions to quickly identify the PRs that are related to software defects.

The PRs and their corresponding before/after fixed version information that had been closed and merged into the master branch that contained the keywords "bug", "fix", "defect" in the title and label were filtered out and stored in the database. The fact that the PR has been closed and merged means that the defect has been located and fixed by the project developer, and thus their defect information and defect code are relatively clear and suitable for collection.

3) Remote repository data collection

Remote repository data collection mainly refers to the project information collection and defect related PRs information collection. With the help of the API provided by GitHub, the GitHub remote repository can be connected to and the project name and owner name obtained in the project selection can be collected.

Project information collection: Since the same piece of code may cause a defect in one piece of software but be correct in another project, the basic information of the project software as the background information of the defect needs to be collected to describe the defect more comprehensively. The project information collection process mainly collects the project description, project label, project programming language, and other basic information. This basic information can describe background information, such as the main functions and programming languages of the project software, as the contextual information of software defects.

Defect related PRs information collection: The collection of defect-related PRs is mainly to obtain the title, label, body, and other description information directly related to defects of the PR as well as the ID of the version (it is called SHA-1 in GitHub) before and after the repair used to locate the defect file and the file path changed between the versions. To determine whether the modified files were code files, we created a map of the suffixes of the various programming languages and their code files by identifying the suffixes of the file paths and excluding non-code defects that were caused by other files, such as configuration files or resource files. At the same time, we eliminated the defects caused by the test file by identifying the keywords of the file path. After the above-stated steps, we obtained the software defect data caused by the code.

4) Local repository data collection

The fourth step of data collection is to collect the corresponding defect code files and other data from the local repository according to the defect information obtained from the remote repository. The git repository tool is used to clone the project repository to the local location according to the git address collected and then obtain the code file and submission information of the corresponding version in the repository according to the version ID that was collected.

C. Software defect code recognition

In the previous step, we were able to obtain different files between the two versions. The code differences before and after the defect repair need to be identified and the code file needs to be tailored to locate the specific location of the defect. Fix-inducing change code is the code that was changed during defect fixing. We assumed that the code of fix-inducing change is the code that introduces defects. Next, we only need to compare the code file before and after modification and extract the modified part, and then the defect code can be cut out and the defect to the code line can be located. This kind of defect location accuracy is higher than the traditional defect location that takes the class as the granularity. We believe that this will be beneficial to improve the effectiveness of SDP and other related research.

Abstract syntax tree (AST) is often used in the research of code semantic mining in software fields, such as defect predict[6]–[8] and code differencing[9]. An abstract syntax tree is a tree-based structure used to represent the syntax and semantics of source code, with each node representing a syntax structure. A simple AST that is parsed from the Java source code is shown in Fig. 5.

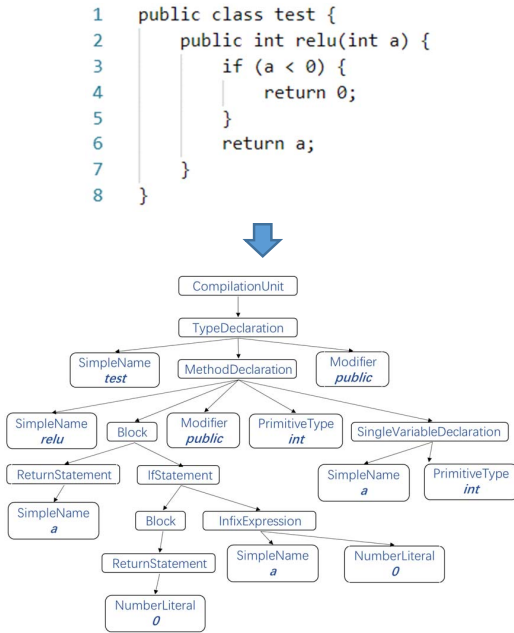


Fig. 5 AST example

To compare the differences, the open-source code scanning tool is used to convert the difference code files into abstract syntax trees, and the open-source syntax tree comparison tool is

used to compare the differences between the two abstract syntax trees, cut out the different tree structure, and obtain the difference code.

The code of a module before and after defect repair can be collected when using this method to collect defect data. By taking the code before repair as defect data and the code after repair as non-defect data, the ratio between defect data and non-defect data can be strictly equal to 1:1, which solves the common problem of class imbalance in the software defect dataset.

D. Topic extraction of contextual information

The main objective of the topic extraction of contextual information is to extract the topic words after integrating the software project information collected in the previous steps with the defect information of the PR. To describe the type and environment of the defect, ten words are extracted from hundreds of words or from additional background information corresponding to a defect.

After comparing various NLP technologies, we decided to use Latent Dirichlet Allocation (LDA)[10] for topic extraction. This model is a probabilistic model generated by a corpus, which is an unsupervised machine learning technology belonging to a Bayesian model. As shown in Fig. 6, the model treats documents as a random mixture of potential topics, each of which is a random mixture of different words assuming that their prior distributions conform to the Dirichlet distribution. Shi et al. proposed a solution method for topic analysis based on this model. The Bayesian inference method was used to obtain the posteriori distribution of document topics for the Dirichlet-multi conjugate model, and then a Gibbs sampling algorithm was used to obtain the topic distribution in the document and the word distribution in the topic.

The advantages of using this method for text processing are as follows: 1) The probabilistic model of the document is uniform and converges quickly. 2) The document probability value has nothing to do with the specific document and does not need to be based on the corpus of the specific software defect related documents. 3) It is not easy to overfit the model with the increase of documents and the increase of parameters.

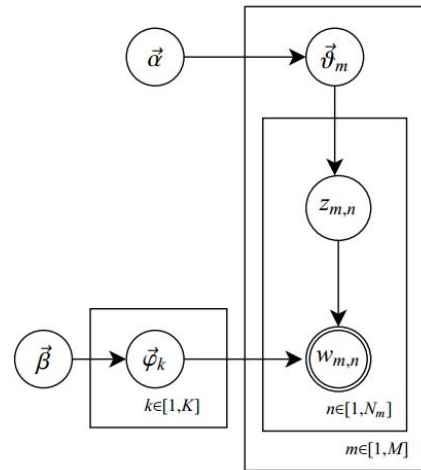


Fig. 6 Latent Dirichlet Allocation

Firstly, defect related information of the PR corresponding to defect data, such as the title, label, main content, and the software information such as the name, label, and description of the software, is integrated to generate shallow background information. Then, the shallow background information is pre-processed using word segmentation, removing the word stop, and making the word stem. The LDA model is constructed after preprocessing, and the shallow background information is input into the LDA model for the solution to realize the extraction of topic words. Therefore, the top ten words of the topic with the highest probability of output distribution are the output result. They represent the contextual information of defects.

E. Software defect data collecting system

By combining the software defect code and background information obtained in the previous two steps, a pair of software defect data containing semantic and contextual information can be obtained. However, the amount of software defect data required by SDP and other applications is very large. Therefore, this section describes the prototype of the automatic defect data collection tool that was developed based on GitHub. In addition, a B/S framework was established for the automatic software defect data collection system. The overall architecture of the system is shown in Fig. 7.

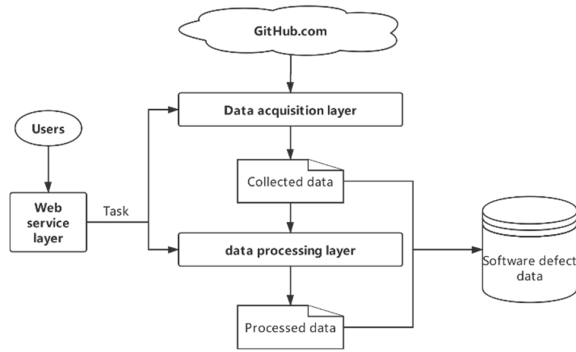


Fig. 7 System architecture

This system mainly consists of the following four parts: web service layer, data acquisition layer, data processing layer, and defect database. The user makes a task request to the server through the web service layer, conducts task scheduling through the task dispatcher, and distributes different kinds of tasks to different modules for processing. The data acquisition layer is mainly responsible for data acquisition of the remote and local repository. The data processing layer is mainly responsible for the identification of defect code and the topic extraction of contextual information as well as the processing of user's requirements for data addition, deletion, examination, and modification. The data transmission and storage between them are realized through the database.

Through the data defect collection system, the project list can be batch imported and automatic defect data collection can be carried out for the project, which greatly improves the efficiency of data collection and truly meets the needs of a large number of software defect data.

IV. VALIDATION

To validate the proposed method, we put forward the following two research questions (RQs) and conducted relevant experiments. We answered the research questions and validated the proposed method by analyzing the experimental results.

A. Research questions

The three RQs are given below.

1) *RQ1: Is the proposed data acquisition method efficient enough?*

The method proposed in this paper collects a large amount of software data and builds defect datasets to support the study of software reliability based on machine learning; therefore, the amount of data to be collected might be huge, which requires the efficient collection of software defect data.

2) *RQ2: Is the data collected by using our proposed data collection method valid?*

The correctness of the data is also a concern; whether the ratio of noise in the collected data is small enough is an important index to judge the validity of the data.

In addition, the proposed method collects software defect data containing code semantics and contextual information of software defects to support software reliability research such as cross-project SDP (CPDP). Therefore, it is an important question whether the data collected by our proposed method can effectively train the SDP model and improve the performance of the SDP.

3) *RQ3: Does the proposed approach outperform other major approaches?*

Some researchers have proposed software defect data collection methods based on OSS. We compare this method with other mainstream software defect data methods. Among the mainstream methods, Kapur's software defect data acquisition method based on Bug report portal and Russell's rule-based static analysis tool labeling defect data are representative.

B. Validation settings

1) Data collection efficiency verification

To verify the efficiency of the proposed collection technology, we first determined the collection target items. To facilitate further validation, we selected over 1000 Java software projects on GitHub fork and filtered out the ones that were not suitable as collection targets. We selected about 587 Java projects as software defect data acquisition targets.

Java was used to build an automated software defect data acquisition platform based on the proposed method and automatically call Python to implement data processing processes, such as subject word extraction. The project was deployed to the Tomcat server and ran continuously until all the defect data of the target project were collected and processed.

2) Validation of the dataset

In the validity test of the dataset, it was mainly divided into the following two parts: whether the data is correct and whether the dataset can be applied in SDP.

a) Correctness validation

To verify the correctness of the data, we conducted a manual review on the collected data, excluding the data with wrong labels (abnormal points), data that could not be applied to defect prediction (outliers), and other noise data. Then, the noise data were classified, the cause of the noise generation was determined, the amount of noise data was counted, and the signal-to-noise ratio (SNR) was calculated.

b) Application validation

In this section, the software defect dataset established in the previous section was used to train the SDP model. By using this model to predict software defects, the quality and effectiveness of the dataset are evaluated according to its f-measure, AUC value, and other evaluation metrics.

Evaluation metrics: Here we provide a brief description of the evaluation metrics. As shown in Fig. 8, the confusion matrix was counted by the actual labels and labels.

	Predicted negative	Predicted positive
Actual negative	true positive	false positive
Actual positive	false negative	true negative

Fig. 8 Confusion matrix

The formulas for calculating the precision and recall are as follows:

$$Precision = \frac{true\ positive}{true\ positive + false\ positive} \quad (1)$$

$$Recall = \frac{true\ positive}{true\ positive + false\ negative} \quad (2)$$

$$F1 = \frac{2 \times (Precision + Recall)}{Precision \times Recall} \quad (3)$$

Receiver operating characteristic (ROC) curves are a popular way to visualize the trade-off between true positive rate and false-positive rate. In practice, the area under a ROC curve (i.e., AUC) is an excellent measure for summarizing the curve in a single value.

Model: An isomorphism network based on graph classification[11] was used for SDP. This method parses the defect code in the defect data into a subtree of the abstract syntax tree, and each node in the tree contains the node type, code content, and other attributes. The defect contextual information in the dataset is transformed into vector data added to the defect subtree attributes. Finally, the adjacency matrix and attribute matrix are used to represent the defect subtree and input into the graph isomorphism network (GIN) as the training data. The hyper-parameters setting are given in Fig. 9.

Model	Learning rate	N layers	Hidden size	Dropout
GIN	0.0001	4	256	0.5

Fig. 9 Hyper-parameters setting

C. Results and analysis

1) Data collection result

Under the unattended condition, the software defect collection system successfully collected a total of 40,000 pairs

of software defect data from 587 Java software projects after running for about 50 hours. After separating the data into defect samples and non-defect samples according to the code before and after repair, a Java software defect dataset consisting of a total of 80,000 software defect data that solves the class imbalance problem could be obtained. By contrast, the NASA dataset, one of the most frequently used software defect datasets in the CPDP, contained data from only 13 projects, with a serious imbalance.

According to the above-mentioned results, we believe that the software defect data acquisition system developed by us can automatically collect data under unattended conditions. At the same time, the collecting speed of this method is much higher than that of the manual method, and the collection efficiency basically meets the application requirements. Moreover, based on defect reports, the amount of data that can be collected is much larger than that of other defect data collecting methods.

2) Analysis and denoising of the dataset

This section describes the correctness verification of the dataset.

Due to the large amount of data that was collected, to facilitate the correctness verification and the next application verification, the Java code defects caused by a single code file were screened out from the dataset to form the Java single file software defect dataset (JSFDD). Java single-file software defect data refers to the defect data that only modified one Java file before and after the defect repair except for the explanatory document (.doc, .txt, .md, etc.).

The JSFDD we created contained 3407 pairs of software defect data from 307 projects. The amount of distribution of software defect data pairs contained in each project in the dataset (a pair of defect data consisting of a defect sample and its corresponding non-defect sample) was calculated. The statistical results are shown in Fig. 10.

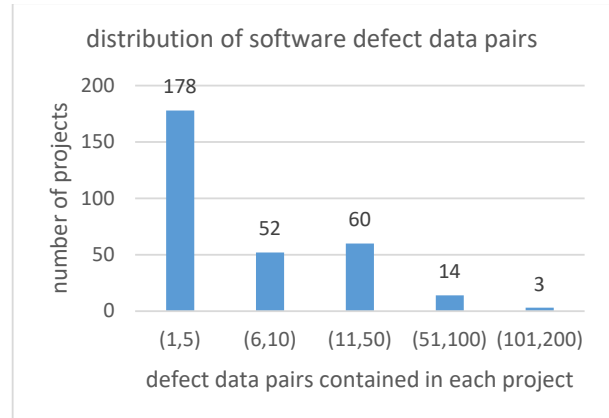


Fig. 10 Distribution of software defect data pairs

As shown above, the number of software defect data pairs contained in most projects was less than 50, more than half of the projects contained less than five pairs of software defect data, only three projects contained more than 100 pairs of defect data, and there was no software containing more than 200 pairs of data. This indicates that the software types covered by defects in this

dataset were wide, the distribution of defects was not centralized, and there was basically no problem of dataset quality caused by the data feature set of a certain software or software type.

In the process of analyzing the JSFDD, we found that there was some noise in the dataset that did not meet the requirements of software defect data, and thus, to achieve the effect of manual denoising, it was necessary to clean the dataset. There are two main causes of noise: fixing defects through refactoring, and realizing other functions while fixing defects. Both of these noises cause the number of lines of modified by the code to be much larger than the actual defect code, which requires filtering of samples of such modified code. An example of some of the noise we removed during the data cleaning process is shown in Fig. 11. After analyzing a total of 3407 defect data, we found 36 pairs of noise data, which accounted for about 1% of the total data. According to the formula, it can be calculated that the SNR of JSFDD was close to 20db, which is a good level.

Noise Type	Example		The specific reason
	Project	Data ID	
Modify Java files that are not functional code	mockito	7dd3c17471abd131174f197f6d98cd3fce5a9af5	Only changed the src/main/Java/org/mockito/mockito. Java. This file is a javadoc file for generating project descriptions, not software function code files.
		7dd3c17471abd131174f197f6d98cd3fce5a9af5	
		7dd3c17471abd131174f197f6d98cd3fce5a9af5	
		0ac384093b1390b4762948a4d25353e36471c60e	
Code refactoring	processing	9d6a09a00e93e0395ba3abdc8bed0832786f5c1	Fixed the bug by refactoring the code
	Arduino	e518050b288843e6027f22ca4e6ebaf440ead759	Code and features are refactored in addition to bug fixes
Contains code changes unrelated to the defect	orientdb	99768b7c522b8c5aac8d1e015b36d74f189fa6f7	also did adaptation between formatting and Java while fixing the defect
	libgdx	8444079ed9b8c11f4793c05362844b43d8eb2fec	Type was modified while the defect was fixed

Fig. 11 Example of noise

The data we collected had a high SNR, which proves that the data had a high enough accuracy. After collection, to improve the quality of the dataset, the dataset could be further processed and de-noised according to the requirements.

3) Software defect prediction experiment

The JSFDD established in the previous section was randomly divided into a training set and test set at a 5:1 ratio. After training the convolutional neural network with the training set data, SDP was carried out on the training set and the test set, respectively, and evaluation metrics were calculated, such as prediction accuracy. The results are shown in Fig. 12.

The variation trend of cross-entropy loss and the variation trend of training set accuracy during iteration are shown in Fig. 13 and Fig. 14.

Train Accuracy	Test Accuracy	Precision	Recall	F-measure	AUC
0.898	0.878	0.911	0.885	0.987	0.895

Fig. 12 Software defect prediction experiment results

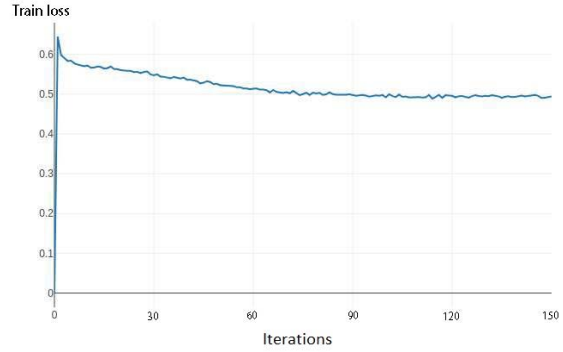


Fig. 13 Train loss

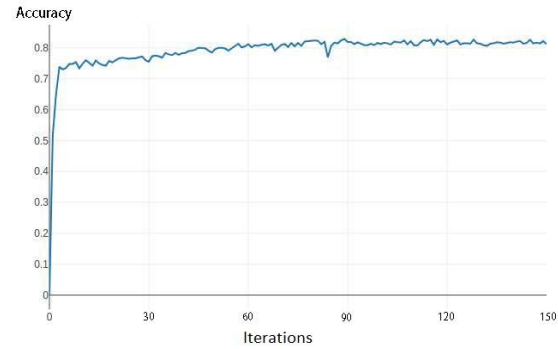


Fig. 14 Accuracy of the training set

By analyzing Fig. 12, it can be seen that both the Train Accuracy and Test Accuracy of SDP by using the training model of software defect data collected by the method we proposed reached above 0.85.

The results show that the model achieves good SDP results, which indicates that the software defect dataset used by the model is of good quality and sufficient to support the SDP technology based on deep learning. This further verifies that the software defect data collection method proposed by us based on GitHub has strong practical significance and can provide data support for software reliability research based on deep learning.

4) Comparison with other methods

Through the integration of other factors, such as data collection efficiency and dataset quality and quantity, we compared this method with two other mainstream methods proposed by Kapur and Russell. The results are shown in Fig. 15.

Since Kapur used a Bug tracking portal based collection method with more detailed report support for defects, it is not inferior to our proposed method in terms of SNR whether it

contains background contextual information and defect type coverage. However, Kapur's method has a fatal defect, that is, in practical application, the number of projects using Apache and other Bug tracking portal is very limited, and thus the amount of defect data that can be collected is difficult to meet the actual needs.

In contrast, Russell's approach used rule-based static analysis tools to scan and flag defects in software code, and thus it targets almost all OSS, and slightly outperforms our approach in terms of the number of items that can be collected. However, using a rule-based static analysis tool to flag defects means that his method only flags defects in software code that violate the rules set by the tool (mainly syntax errors) and ignores defects introduced for other reasons, such as semantics. Because this method only covers some defect types, the SNR is very high and the background contextual information is not described.

At the same time, compared with the methods proposed by us, these two methods have the characteristics of not storing the semantics of the presentation code and not solving the problem of unbalanced classes of software defect datasets.

Feature	Our method	Russell	Kapur
data source	GitHub repository	GitHub repository	GitHub repository managed by Bug tracking portal
Defect label method	Pull Requests	Static analysis tool	The bug report
Amount of project	★★★	★★★☆	★
Contextual information	✓	×	✓
Code semantic	✓	×	×
Signal to noise ratio	★★	★	★★★
Defect type coverage	★★★	★	★★★
Class imbalance	✓	×	×
Collection efficiency	★★	★★★	★

Fig. 15 Comparison

As shown in Fig. 15, the proposed method has the advantages of sufficient and comprehensive sources, efficient and convenient collection, and rich and accurate data.

V. RELATED WORK

Software defect data: In previous software reliability research, some exposed software defect datasets based on measurement metrics were mainly used. Various metrics are developed such as McCabe's cyclomatic complexity[12], Halstead complexity measures[13], CK[14], MOOD[15], Structure metrics of Software Network[16][17], etc. NASA's dataset consists of data from 13 publicly available and widely used software. In addition, because Eclipse is a widely used IDE software, Zeller[18] mined the defect data of its various versions to form the Eclipse dataset. There are other public datasets, such as the PROMISE version of the NASA dataset. Shepperd[19] analyzed the two versions of NASA datasets and found quality problems, such as data loss, abnormal values, and repeated instances, and the version difference had a significant impact on the SDP results. Wang[20] spectral cluster-based Feature Quality coefficient to measured features in PROMISE dataset and found that most of them are helpless for SDP. Hence, it is an important research topic in the field of software reliability to find a method to establish high-quality software defect datasets. Different from the above dataset, we also identify and crop the

software defect code to make the defect code in the data accurate to the line. At the same time, we also solve the problem of class imbalance in the software defect dataset, and the dataset quality and learning value are high.

Defect data collection: In past defect dataset establishment, the method of collecting defect data was calculating the measurement metrics of each version of the software code and manually analyzing each version of the code and labeling the defects. With the introduction of machine learning and deep learning into the field of software reliability research, the efficiency of this method of manually collecting and labeling data has completely failed to meet the requirements of model training on data volume. Russell[3] used rule-based static code analysis tools, such as Flawfinder, to label defects in open source projects and proposes a method of constructing defect datasets based on all OSS. Based on the defect report of Apache and other defect tracking systems, Kapur[2] labeled each version of code in the corresponding open source repository on GitHub and proposed a method to build defect dataset based on the defect tracking system. Different from the above method, the method we proposed is based on GitHub PRs workflow to label defects, which not only collects a large amount of data efficiently but also ensures the quality of defect data.

VI. CONCLUSION

This paper proposed an efficient software defect data collection method. The proposed method is based on the OSS hosting platform GitHub and its PR workflow pair to find software defect data. The defect code was located using the AST comparison method, and the contextual information of the software defect was extracted with the NLP method. The software defect data in our dataset not only contains the specific defect code and the contextual information of the defect but also solves the general class imbalance problem in the software defect dataset.

Through the data collection process and dataset analysis, as well as the application in CPDP, the software defect data collection method proposed in this paper was validated. Validation results show that the proposed method can automatically collect a large number of defect data and provide data support for software reliability research, such as SDP.

Compared with the traditional method, the advantages of the proposed are summarized below.

1. We used PRs on GitHub as the basis for searching and labeling software defects and conducted automatic collection, which saves a lot of manpower and time for users to label data. At the same time, it covers all kinds of defects caused by rules and semantics and ensures the accuracy and comprehensiveness of defect labeling.
2. Due to the large number of projects on GitHub that use the merge request workflow, the amount of software defect data that can be collected is much larger than the collection methods based on Apache and other defect tracking portals.
3. We cut defect codes through AST to improve the granularity of data to the line of code, which greatly improves the accuracy compared with traditional datasets.

4. By taking the code before defect repair as a defect sample and the code after defect repair as a defect-free sample, the proportion of the two types of samples is strictly 1:1, and the common class imbalance problem is solved.

5. We extracted the subject words of software defect information and software function information to provide the contextual information for the defect.

6. We create a public defect dataset based on GitHub through proposed method. Our GHPR dataset of cross-projects defects is available in a GitHub repository (https://github.com/HickeyHsu/GHPR_dataset).

Future work will address how to effectively clean and denoise large amounts of software defect data, improve the quality of the dataset, and how to tailor software defect code to consider unmodified but actually defect-related code near the modified code.

ACKNOWLEDGMENT

The author would like to thank all reviewers anonymous for their constructive suggestions on this article. At the same time, the work cannot be completed without the support of SDR research group of Beihang University.

REFERENCES

- [1] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, "A survey of machine learning for big code and naturalness," *ACM Comput. Surv.*, vol. 51, no. 4, 2018.
- [2] R. Kapur and B. Sodhi, "Estimating defectiveness of source code: A predictive model using GitHub content," 2018 [Online]. Available: <http://arxiv.org/abs/1803.07764>
- [3] R. Russell et al., "Automated Vulnerability Detection in Source Code Using Deep Representation Learning," *Proc. - 17th IEEE Int. Conf. Mach. Learn. Appl. ICMLA 2018*, no. ML, pp. 757–762, 2019.
- [4] E. J. Weyuker, T. J. Ostrand, and R. M. Bell, "Do too many cooks spoil the broth? Using the number of developers to enhance defect prediction models," *Empir. Softw. Eng.*, vol. 13, no. 5, pp. 539–559, 2008.
- [5] Q. Song, Z. Jia, M. Shepperd, S. Ying, and J. Liu, "A general software defect-proneness prediction framework," *IEEE Trans. Softw. Eng.*, vol. 37, no. 3, pp. 356–370, 2011.
- [6] T. Ben-Nun, A. S. Jakobovits, and T. Hoefler, "Neural code comprehension: A learnable representation of code semantics," *Adv. Neural Inf. Process. Syst.*, vol. 2018-Decem, no. NeurIPS, pp. 3585–3597, 2018.
- [7] H. K. Dam et al., "A deep tree-based model for software defect prediction," 2018 [Online]. Available: <http://arxiv.org/abs/1802.00921>
- [8] S. Wang, T. Liu, J. Nam, and L. Tan, "Deep Semantic Feature Learning for Software Defect Prediction," *IEEE Trans. Softw. Eng.*, vol. 5589, no. c, pp. 1–26, 2018.
- [9] J. R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus, "Fine-grained and accurate source code differencing," *ASE 2014 - Proc. 29th ACM/IEEE Int. Conf. Autom. Softw. Eng.*, pp. 313–323, 2014.
- [10] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet Allocation David," vol. 3, pp. 993–1022, 2003 [Online]. Available: <http://www.jmlr.org/papers/volume3/blei03a/blei03a.pdf>
- [11] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?," 2018.
- [12] T. J. McCabe, "A Complexity Measure," *IEEE Trans. Softw. Eng.*, 1976.
- [13] M. H. Halstead, *Elements of Software Science (Operating and Programming Systems Series)*. New York, NY, USA: Elsevier Science Inc., 1977.
- [14] S. Chidamber and C. F. Kemerer, "A Metric suite for object oriented design," *IEEE Trans. Softw. Eng.*, 1994.
- [15] V. R. Basili, L. C. Briand, and W. L. Melo, "A validation of object-oriented design metrics as quality indicators," *IEEE Trans. Softw. Eng.*, 1996.
- [16] Y. Yang, J. Ai, and F. Wang, "Defect Prediction Based on the Characteristics of Multilayer Structure of Software Network," in *2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, 2018, pp. 27–34.
- [17] J. Ai, W. Su, S. Zhang, and Y. Yang, "A Software Network Model for Software Structure and Faults Distribution Analysis," *IEEE Trans. Reliab.*, vol. 68, no. 3, pp. 844–858, 2019.
- [18] T. Zimmermann, R. Premraj, and A. Zeller, "Predicting defects for eclipse," *Proc. - ICSE 2007 Work. Third Int. Work. Predict. Model. Softw. Eng. PROMISE'07*, 2007.
- [19] M. Shepperd, Q. Song, Z. Sun, and C. Mair, "Data quality: Some comments on the NASA software defect datasets," *IEEE Trans. Softw. Eng.*, vol. 39, no. 9, pp. 1208–1215, 2013.
- [20] F. Wang, J. Ai, and Z. Zou, "A Cluster-Based Hybrid Feature Selection Method for Defect Prediction," in *Proceedings - 19th IEEE International Conference on Software Quality, Reliability and Security, QRS 2019*, 2019, pp. 1–9.